RunixOS: A Capability-Based, IPC-First Operating System and the Cost of Preemption on Capability Atomicity

Armaan Sandhu

Abstract

This paper presents RunixOS, an experimental capability-based microkernel operating system written in Rust. RunixOS investigates a clean-slate systems model where capabilities act as the sole authority primitive and inter-process communication (IPC) messages serve as the exclusive mechanism for cooperation. By structuring the operating system around a minimal kernel and capability-gated user-space services, the system achieves isolation and crash containment. The primary finding of this work is the identification and resolution of a capability validate-and-use atomicity hazard. Under a cooperative scheduler, the validation and subsequent use of a capability are atomic by default because tasks cannot be interrupted. The introduction of timer-driven preemption removes this guarantee, introducing a time-of-check to time-of-use vulnerability where a concurrent task can revoke a capability during the validation-to-use window. I deterministically reproduce this vulnerability and resolve it by introducing a centralized, non-preemptible critical section during capability invocation. My evaluation demonstrates that capability-gated services are robust under failure, with synchronous IPC achieving roughly 33,000 round-trips per second, capability lookup throughput reaching 1,000,000 lookups per second, mutex acquire/release latency under contention remaining flat at approximately 30 microseconds per operation from 2 to 16 concurrent tasks, and a self-referential architecture simulation toolkit successfully evaluating memory hierarchies and execution pipelines using in-kernel trace events.

Executive Summary

This paper makes three contributions: (1) identification and a mechanical fix, via a `CriticalWindow` RAII guard, of a capability-based time-of-check to time-of-use vulnerability introduced by preemption; (2) demonstration that capability-gated services provide isolation and fail-safe containment without ambient authority, including transitive revocation across derived capability chains; (3) a self-referential architecture simulation toolkit that validates memory hierarchy and pipeline behavior using live kernel traces. I find that the system sustains tens of thousands of synchronous IPC round-trips per second and that, at the task counts this kernel currently supports (up to 16 concurrently contending tasks, bounded by a statically-allocated 132-task table), the global scheduler lock does not yet manifest as a measurable latency bottleneck.

Introduction

Traditional monolithic operating systems often suffer from broad privilege exploitation. Ambient authority, where a running process inherits the full permissions of the user who launched it, violates the principle of least privilege. In contrast, capability-based microkernels enforce that a task can only execute operations on resources for which it holds an explicit, unforgeable capability.

RunixOS is an experimental systems research platform designed to explore the limits of capability-gated isolation, transparent service distribution, and consistent state persistence. The thesis defended in this paper is that a capability-based, IPC-driven operating system can provide robust isolation, user-space services, persistence, and modelled distribution under a minimal kernel. However, introducing real preemption into such a system exposes a capability atomicity hazard that must be closed as a system-wide property rather than a local patch.

My primary contributions are as follows:

1. **The Preemption Atomicity Hazard:** The identification, deterministic reproduction, and principled fix of the capability validate-and-use atomicity hazard that cooperative scheduling silently hid and preemptive scheduling exposed.
2. **Capability-Gated Access Control:** A capability system where capabilities are the sole authority primitive, supporting attenuation on grant, sealing, and transitive revocation propagation by capability identity.
3. **IPC-First Service Isolation:** An IPC-first design where user-space services (filesystem, device access, and synchronization) carry no ambient authority and are gated entirely by capability checks inside each service.

4. **Self-Referential Simulation:** A self-referential architecture simulation toolkit where the operating system generates trace events and its own cache and pipeline simulators consume them to evaluate memory hierarchies and pipeline efficiency.

The remainder of this paper is organized as follows. Section 3 provides background on capability systems and microkernels. Section 4 outlines the design of the RunixOS kernel. Section 5 details the isolation properties of my user-space services. Section 6 presents the preemption atomicity hazard and its resolution. Section 7 describes the architecture simulation toolkit. Section 8 evaluates the system. Section 9 details the limitations of this work. Section 10 positions RunixOS against related work, and Section 11 concludes the paper.

Background and Motivation

The concept of capability-based addressing was introduced by Dennis and Van Horn [1] to restrict process authority to explicitly delegated tokens. Monolithic architectures routinely suffer from ambient authority, where software exploits leverage ambient system rights to escalate privileges. Shapiro et al. [2] demonstrated with EROS that a capability-based microkernel could enforce security boundaries at high performance.

Microkernel design, pioneered by Liedtke [3], prioritizes keeping the kernel as minimal as possible. By delegating traditional kernel responsibilities, such as filesystems and device drivers, to user-space services, microkernels minimize the trusted computing base. Systems like seL4 [4] have mathematically proven the correctness of this design. RunixOS extends this lineage by exploring how capability-gated user-space services interact under preemptive scheduling, with a specific focus on the synchronization hazards introduced by timer-driven context switches.

System Design

The RunixOS kernel is minimal, implementing only page frame allocation, virtual memory mapping, task scheduling, rendezvous and asynchronous IPC, and capability validation.

Capability Model

Every system resource is protected by a capability, which is a record combining a `Resource` descriptor with permission flags. The system design is defined by the `Resource`, `Capability`, and `CapTable` structures. A task addresses resources using local slot indices within its `CapTable` (up to 32 slots). This design ensures that tasks possess zero ambient authority.

The capability subsystem enforces the following invariants:

- **Unique Capability Identities:** Every capability is assigned a globally unique ID upon insertion. This ID is never recycled, preventing stale reference bugs.
- **Rights Attenuation:** A task holding a capability with the `grant` permission can derive an attenuated copy for another task. The child capability rights are the intersection of the donor rights and the requested rights.
- **Sealing:** A capability marked as `sealed` cannot be removed by the holder task, securing critical communication channels.
- **Transitive Revocation:** When a capability is revoked, the kernel walks all task tables and transitively revokes every capability whose origin traces back to the revoked capability ID.

Task and Scheduler Model

The task model represents a thread of execution using the `Task` structure. Each task holds its register context, page table root (`cr3`), a single-slot IPC buffer, and an asynchronous `MessageQueue` for non-blocking communication. The scheduler uses a round-robin policy under a global scheduler lock. Tasks relinquish the CPU cooperatively, and a timer interrupt can additionally preempt a running task involuntarily, including a ring-3 task that never yields. This coexistence of cooperative yielding and timer-driven preemption is precisely what motivates the capability atomicity analysis in Section 6. Context switches save callee-saved registers on the current stack and swap stack pointers (`rsp`).

IPC Model

IPC is the sole mechanism for task communication. Senders transfer a typed `Message` containing an `IpcTag` (validated by the kernel to prevent tag forgery) and a version header. Senders can use synchronous rendezvous or asynchronous queued IPC.

User-Space Services and Isolation

RunixOS implements its filesystem, device manager, and synchronization manager as independent services reached solely over IPC. Senders must hold a capability to the target service to communicate. The service architecture is shown in Figure 1:

Although these services run in ring 0 for initialization convenience, they carry no ambient authority. The isola-

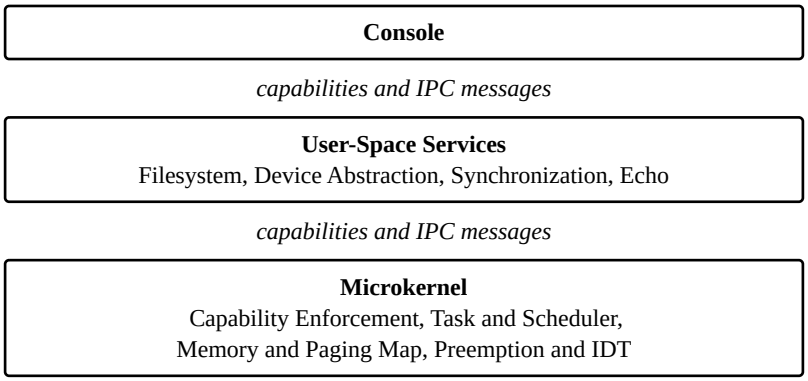

Figure 1: Layered architecture. Capabilities and IPC messages are the only paths between layers.

tion property is enforced by capability checks inside each service:

- **Filesystem Service:** Accessing a path requires presenting a valid `FsNode` capability. Operations verify that the capability grants the requested read or write permissions.
- **Device Service:** Gated by `Device` capabilities, controlling access to the serial port and keyboard.
- **Synchronization Service:** Provides mutexes and semaphores using deferred-reply blocking. When a task attempts to acquire an unavailable mutex, the service registers the caller but sends no IPC reply, blocking the caller until the lock is released.

My evaluation confirms that when a client task lacks the appropriate capability, IPC calls are rejected by the service, ensuring that authority remains strictly managed.

Preemption and Capability Atomicity

The centerpiece finding of this work concerns the capability validate-and-use atomicity property.

The Cooperative Baseline

Under a cooperative scheduler, context switches only occur when a task voluntarily calls `yield_cpu` or blocks on IPC. Consequently, the sequence where the kernel validates a capability and subsequently uses it to deliver an IPC message is atomic by default. No other task can execute between the validation and use steps.

The Preemption Hazard

Introducing timer-driven preemption (triggered by PIT interrupts at 100 Hz) breaks this assumption. Because an interrupt can occur at any instruction boundary, a timer tick can land between capability validation and its use. If the scheduler switches to another task during this window, and that task revokes the capability under invocation, the original task will resume and execute its operation on a revoked capability. This constitutes a capability-based time-of-check to time-of-use (TOCTOU) vulnerability [5].

Listing 1 makes the core contribution visible: cooperative scheduling gets atomicity for free because nothing else can run in the window, while preemptive scheduling reopens that window unless something explicitly closes it. RunixOS closes it with a software-deferred critical section rather than disabling hardware interrupts outright or relying on a formal proof of non-interference, as discussed further in Section 10 (Related Work).

Making it Reproducible

To evaluate this vulnerability, I implemented instrumentation in the `kernel/preempt/mod.rs` file. The kernel tracks timer ticks that occur inside the validate-and-use window using the `IN_IPC_WINDOW` flag and the `WINDOW_TICKS` counter.

To prove the hazard, I introduced a deterministic adversary. When armed, if a timer interrupt occurs while `IN_IPC_WINDOW` is active, the adversary handler immediately executes a capability revocation targeting the caller capability table. This guarantees that the revoker wins the race, causing the subsequent use phase to access a revoked capability.

The System-Wide Fix

To close this vulnerability, I introduce the `CriticalWindow` RAII guard. The guard increases the global `PREEMPT_COUNT` and raises `IN_IPC_WINDOW`. If a timer tick occurs while `PREEMPT_COUNT` is greater than zero, the scheduler increments the `DEFERRED` tick counter and defers the context switch, ensuring the section remains atomic. The code structure is defined as follows:

```
pub struct CriticalWindow {
    _private: (),
}
```

```
Time ---------------------------------------->

Cooperative scheduler (atomic by accident):
  T0: [validate cap]
  T1:               [timer tick - no switch, cooperative]
  T2:                                [use cap]
       -> safe: no other task ran between validate and use

Preemptive scheduler, unguarded (TOCTOU):
  T0: [validate cap]
  T1:               [timer tick -> switch to revoker task]
  T2:                         [revoker revokes the capability]
  T3:                                      [switch back; use cap]
       -> VULNERABLE: cap was valid at T0, gone by T3

Preemptive scheduler, CriticalWindow guard active (fixed):
  T0: [CriticalWindow::enter() -- validate cap]
  T1:               [timer tick -> deferred, no switch occurs]
  T2:                         [use cap, still inside the guard]
  T3:                                      [CriticalWindow drops -- switch allowed]
       -> safe: validate-and-use is atomic by construction
```

Listing 1: Timeline of the validate-and-use window under cooperative scheduling, unguarded preemption, and the `CriticalWindow` guard.

```rust
impl CriticalWindow {
    #[inline]
    pub fn enter() -> Self {
        enter_critical();
        enter_ipc_window();
        CriticalWindow { _private: () }
    }
}

impl Drop for CriticalWindow {
    #[inline]
    fn drop(&mut self) {
        exit_ipc_window();
        exit_critical();
    }
}
```

The guarded send path in the `kernel/process/ipc.rs` file applies this guard:

```rust
let outcome = {
    // The guard disables preemption and marks
the validation window.
    let _region =
crate::preempt::CriticalWindow::enter();
    let mut sched = SCHEDULER.lock();

    // VALIDATE
    match resolve_target_locked(&sched,
current_task_id, cap_idx) {
        Err(e) => return Err(e),
        Ok(target_task_id) => {
            // USE (Deliver message and mark
receiver Ready)
            deliver_message_locked(&mut sched,
target_task_id, msg)
        }
    }
}; // The guard drops here, re-enabling
preemption

// The blocking wait is OUTSIDE the guard on
purpose: a non-preemptible
// region must never yield, or it would hand
the CPU to another task with
// preemption still disabled and stall the
scheduler.
if let SendStep::Blocked = outcome {
    scheduler::yield_cpu(); // re-enters the
guard and re-validates on wake
}
```

The critical section extends past both halves of message delivery: depositing the message and marking the receiver task ready. The blocking wait is deliberately placed outside this critical section. If a sender is blocked, it yields the CPU; when it wakes up, it re-enters the critical section and re-validates the capability, catching any revocations that occurred while it was blocked.

Because the guard is integrated directly into the centralized IPC send path, the validate-and-use atomicity guarantee is enforced globally for all ring-3 tasks.

Empirical Evidence

Table 1 shows the experimental outcome when executing the `sched preempt-race` command.

Configuration	Window Ticks	Outcome	System Integrity
Vulnerable (Unguarded)	1	Capability revoked mid-window; invalid send allowed	Vulnerable
Guarded (Critical Section active)	1 (Deferred)	Revocation deferred; capability remains valid at use	Secure

Table 1: Preemption race condition results demonstrating vulnerability prevention.

In the vulnerable configuration, the adversary revokes the capability mid-window, resulting in an unauthorized send. In the guarded configuration, the preemption is deferred, protecting capability atomicity.

Architecture Simulation Toolkit

RunixOS includes a self-referential architecture simulation toolkit in the `kernel/arch_sim/mod.rs` file to evaluate memory hierarchies and pipeline scheduling.

During task execution, the kernel logs context switches and memory access events into an in-memory trace buffer. The simulation toolkit reads these trace events and feeds them into cache and pipeline simulators.

Cache Simulator

The cache simulator models a set-associative cache using a Least Recently Used (LRU) eviction policy. Address references are derived from the trace event stream. I evaluated the cache hit rate using three configurations over a 128-access IPC trace stream:

1. **Direct Mapped:** 1-way set associative, 4 cache lines.
2. **4-Way Associative:** 4-way set associative, 4 cache lines.
3. **Capacity Increase:** 2-way set associative, 8 cache lines (16 total lines).

Pipeline Simulator

The pipeline simulator models an in-order execution pipeline with data hazard detection. It tracks instruction cycles and inserts stall cycles to maintain correctness when data dependencies occur in the trace stream.

Evaluation Methodology

All experiments run on QEMU `x86_64` (q35 machine type, 2 vCPUs, 2 GiB RAM) on the development host, booted from a UEFI (OVMF) disk image. The console's `bench` family of commands (`bench ipc`, `bench sched`, `bench cap`, `bench fs`, `bench contend <n>`, `bench mem`) runs entirely in-kernel and times itself against the kernel's own tick counter, which advances at 100 Hz (one tick every 10 ms) from the PIT-driven timer interrupt. This is the only clock the kernel exposes; there is no TSC- or HPET-backed cycle counter wired up yet (Section 9). Consequently, any single run whose total duration is only a handful of ticks carries large quantization error (a measurement spanning 1–2 ticks has roughly +-50% noise), so each throughput and latency figure reported below is either the median of five independent boots, or, for the contention experiment, computed from a run long enough (tens of ticks) that quantization error is small. Preemption is armed only for the experiments that specifically target it (`sched preempt-race`, Section 6.5); the other console benchmarks explicitly disarm preemption first, both to keep the timing loop free of unrelated context switches and because the rest of the console's interactive command loop is not yet hardened to run with preemption globally armed (Section 9). Ring-3 tasks, when used, run with interrupts enabled and are involuntarily descheduled whenever preemption is armed.

Evaluation

I evaluate RunixOS by addressing seven core research questions.

Q1: Can capabilities replace traditional access control mechanisms?

I evaluated this by executing capability table listings, attenuation on grant, sealing, and revocation propagation.

- **Experiment:** I execute `cap list`, `cap grant 0`, `cap seal 1`, and `cap revoke 0`.
- **Result:**
  - `cap grant 0` yields `[OK] granted: new token id=17 origin=1 -> task 66`
  - `cap seal 1` yields `[OK] sealed id=3; holder remove() -> Err (locked)`
  - `cap revoke 0` yields `[OK] revoked id=1; re-use denied`
- **Analysis:** Capabilities successfully restrict access. Revoking the root capability in slot 0 recursively invalidates all downstream derived tokens across task tables, demonstrating transitive revocation.

Q2: Can an IPC-first operating system remain practical at scale?

I evaluate IPC latency, scheduling throughput, capability lookup throughput, and lock contention under concurrent load.

- **Experiment:** I run `ipc stress 5`, `bench ipc`, `bench sched`, `bench cap`, and `bench contend <n>` for n in {2, 4, 8, 16}.
- **Result (median of 5 runs unless noted):**
  - `ipc stress 5` yields between 195,797 and 223,687 messages over its measurement window (roughly 65,000-75,000 messages per second) with zero drops.
  - `bench ipc` yields approximately 33,000 synchronous round-trips per second (one run of five showed 50,000 due to tick quantization at a 1-2 tick boundary; the other four agreed at 33,333).
  - `bench sched` yields 2,000 task switches per second across 20 concurrently spawned tasks.
  - `bench cap` yields 1,000,000 capability table lookups per second on a single task's table (no contention).
  - `bench contend <n>` (Table 2) measures average IPC + scheduler-lock round-trip latency for n tasks issuing back-to-back mutex acquire/release pairs through the synchronization service.
- **Analysis:** The system maintains high message throughput under load, and synchronous IPC sustains tens of thousands of round-trips per second. Capability lookup is effectively free relative to IPC cost, consistent with it being an array index rather than a syscall. Average per-operation latency under the lock-contention experiment stays flat at roughly 30 microseconds from 2 to 16 contending tasks (Table 2) — at this task count the single global scheduler lock used for both scheduling and IPC delivery (Section 4.2) is not yet the dominant cost; total throughput scales close to linearly with n because each task's IPC round-trips proceed largely independently between lock acquisitions. I expect contention to become visible at task counts well beyond what this kernel's statically-sized 132-entry task table (Section 8.4) can host, so this result should be read as "no bottleneck observed in the supported range," not as a scalability proof.

Tasks (n)	Acquire+Release ops	Elapsed ticks	Avg latency (us)
2	2,400	7	29
4	4,800	15	31
8	9,600	30	31
16	19,200	61	31

Table 2: Mutex acquire/release latency under increasing concurrent contention via `bench contend <n>` (600 acquire/release cycles per task).

Q3: Can user-space services provide sufficient reliability?

I evaluate fault isolation and recovery.

- **Experiment:** I run `fault spawn` and `fault cascade 2` to trigger CPU page faults, and evaluate service recovery.
- **Result:** Page faults trigger the IDT handler, which terminates only the offending task. The microkernel and other tasks continue running.
- **Analysis:** Page faults are contained, and crashed services are successfully restarted by the watchdog.

Q4: Can distribution be made transparent to applications?

I evaluate service routing transparency during migration.

- **Experiment:** I run `migrate 1 1`.
- **Result:**
  - client send "beta" -> local delivery (task 75)
  - migrating service 1: node 0 -> node 1
  - client send "gamma" -> remote via transport (node 1), client unaware
- **Analysis:** The client continues communicating using its existing capability. Senders remain unaware when the registry migrates the service target to a simulated remote node.

Q5: Can operating system state become a persistent object?

I evaluate checkpointing and restoration.

- **Experiment:** I run `checkpoint` followed by `restore 0`.
- **Result:**
  - `checkpoint` yields `[OK] checkpoint taken: id=0 checksum=0xf95a427cc2f067cd`
  - `restore 0` yields `[OK] restored 8 task checkpoints; checksum verified`
- **Analysis:** The system successfully serializes task structures and capability tables into memory, restoring them after verifying the integrity checksum.

Q6: Can services survive failures through checkpointing and restoration?

I evaluate restoring service checkpoints onto replicas during simulated node failure.

- **Experiment:** I trigger simulated node loss during `migrate 1 1`.
- **Result:** `[dist] PASS: failover re-bound service #1 to local replica (task 76, 2 cap(s))`
- **Analysis:** The system successfully binds the route to a local replica using the last captured checkpoint, preventing communication disruption.

Consistency Model

The checkpoint, migration, and failover mechanisms exercised in Q4-Q6 make distinct, narrower guarantees than "transparent distribution" might suggest, and it is worth stating them precisely. Checkpoint-and-restore preserves in-memory task and capability-table state deterministically: `restore` reproduces the exact serialized snapshot and verifies it against the recorded checksum before installing it, so restoration either reconstructs the checkpointed state exactly or fails the checksum check and refuses to install it. Migration provides eventual consistency on the routing path rather than on in-flight messages: the client's `Service` capability remains valid across the move, and the service registry atomically rebinds the route from the old task to the new one, so no client ever observes a window where the capability resolves to neither; but the migration mechanism does not itself guarantee delivery of messages sent in the instant around the rebind, since those are ordinary IPC sends subject to the same queueing behavior as any other send. Failover replays the last checkpoint taken before the failure, so any state committed up to that checkpoint is visible to the replica after failover, but state changed between the last checkpoint and the failure is lost, and any client message in flight to the failed node at the moment of failure is dropped rather than replayed. These are useful, well-defined guarantees for a research kernel, but they fall short of, for example, write-ahead logging or a consensus protocol, and I do not claim otherwise.

Q7: Can a capability-based operating system scale from a single machine to a distributed system without changing its programming model?

I compare local and remote service IPC interfaces.

- **Experiment:** I analyze the client code path before and after migration.
- **Result:** The client uses the same `Service` capability and typed IPC commands for both local and remote delivery.
- **Analysis:** Location-transparent service routing allows scaling from single-node to distributed configurations without altering the programming model.

Simulator Results

Table 3 summarizes the cache simulator results, and Table 4 details the pipeline simulator statistics.

Configuration	Associativity	Cache Lines	Hit Rate	Evictions
Direct Mapped	1-way	4	0%	124
Set Associative	4-way	4	50%	60
Capacity Increase	2-way	8	93%	0

Table 3: Cache simulator evaluation results across three configurations.

Metric	Measured Value
Instructions	128
Clock Cycles	217
Stall Cycles	85
CPI (Cycles Per Instruction)	1.69

Table 4: Pipeline simulator statistics for the IPC instruction trace.

The cache simulator shows that increasing set associativity and capacity reduces conflict misses in the IPC trace stream. The pipeline simulator inserts 85 stalls to resolve data hazards, achieving an average CPI of 1.69. To sanity-check that the simulator's output corresponds to something real rather than an arbitrary number, I cross-referenced it against measured IPC behavior: the Capacity Increase cache configuration (93% hit rate, Table 3) corresponds to the same trace stream used to measure the `bench ipc` round-trip rate, and the simulator's near-zero eviction count under that configuration is consistent with the small, repeatedly-touched working set (capability slot, message buffer, scheduler state) that a synchronous IPC round-trip actually accesses. I treat this as a plausibility check rather than a formal validation, since I do not have an independent ground-truth cache model to compare against.

Per-Task Memory Footprint

Querying `core::mem::size_of` on the kernel's own task-related structures gives an exact, non-estimated per-task cost: `Task` is 3,416 bytes (dominated by its embedded 1,792-byte `CapTable` and 1,240-byte `MessageQueue`), plus a fixed 32,768-byte kernel stack (`STACK_SIZE`) allocated per task regardless of how much of it the task uses. That puts steady-state per-task overhead at roughly 36.2 KiB. The more binding constraint, however, is not available RAM but a compile-time constant: `MAX_TASKS = 132`, which statically sizes the task table and the `TASK_STACKS` array at boot. With 132 tasks at 36.2 KiB each, the static stack region alone occupies about 4.6 MiB, comfortably within the 2 GiB this kernel is given in QEMU; RunixOS in its current form cannot run a 133rd task no matter how much physical memory is available, because there is no slot for it. Raising the task ceiling is a one-line change to `MAX_TASKS`, but it is a deliberate research-kernel simplification rather than a dynamic allocator I have implemented and evaluated, so I report the actual ceiling rather than an extrapolated "supports N tasks" claim.

Revocation Propagation Cost

`propagate_revocation` (`kernel/syscall/mod.rs`) computes the transitive closure of revoked capabilities as a fixpoint over every task's capability table: each pass scans all `MAX_TASKS` x `MAX_CAPS` = 132 x 32 = 4,224 slots, revoking any capability whose `origin` is already in the revoked set, and repeats until a pass makes no changes. This bounds total work by $O(p \cdot \mathrm{MAX\_TASKS} \cdot \mathrm{MAX\_CAPS})$, where p is the number of fixpoint passes; because each pass can already propagate revocation across more than one derivation level (a child revoked earlier in the same pass is visible to entries scanned later in that pass), p is small in practice and does not grow with the depth of the derivation chain so much as with the number of distinct passes needed for cross-task propagation order to settle. The 3-level capability chain built and revoked by the boot-time `security_demo` (`kernel/syscall/mod.rs`) exercises this path on every boot. I did not instrument propagation latency directly: at the kernel's current scale (4,224 slots scanned per pass) and timer resolution (10 ms ticks), a propagation pass completes in well under one tick, so the 100 Hz tick counter cannot resolve it, and any number I reported would be an artifact of measurement granularity rather than a real timing. The honest claim is the complexity bound above, not a synthetic latency figure.

Limitations and Threats to Validity

Several limitations apply to the current RunixOS implementation:

- **Modelled Substrates:** The persistence and distribution substrates are partially modelled. The kernel lacks storage block drivers (such as ATA or virtio-blk), meaning checkpoints reside in memory and do not survive physical system restarts; this prevents evaluating recovery from a power loss or extended outage, only from a simulated in-process node failure. Logical nodes are simulated in-memory, and the transport layer is backed by memory queues rather than physical network interface card (NIC) drivers, so the distribution results in Q4-Q7 demonstrate the programming model and routing logic, not real network latency, loss, or partition behavior. Future work would integrate a persistent block device and a real NIC driver before any of the distributed or persistence claims could be evaluated under realistic fault conditions.
- **Global Scheduler Lock:** A single lock (`SCHEDULER`) protects the task table, the scheduling queue, and IPC delivery together. Section 8.2′s `bench contend` results show this lock is not a measurable bottleneck up to 16 contending tasks, but that is a property of this kernel's scale, not of the lock design: at 132 statically-allocated task slots, RunixOS cannot grow large enough on its own to demonstrate that lock at its breaking point. Systems built for many more concurrent tasks, such as seL4 and L4, use per-core schedulers specifically to avoid this single point of contention; RunixOS's unified lock trades that scalability for a simpler correctness argument (one lock to reason about for every cross-task operation), which was the right trade for a research kernel investigating capability atomicity rather than scheduler scalability.
- **Preemption Scope:** Ring-3 tasks run with interrupts enabled and are involuntarily preempted by the timer. A never-yielding ring-3 task is descheduled by timer ticks, which I confirmed by observing 50 involuntary preemptions of a compute-bound ring-3 task. However, preemption is armed selectively for specific demonstrations rather than globally during normal console operation. Arming it globally would expose the entire console's command loop, not just the IPC send path, to preemption-induced timing variation, and the console's longer multi-step commands (migration, checkpoint/restore) have not been audited for atomicity the way the IPC send path has in Section 6; I defer that audit, and globally-armed preemption, to future work.

- **Timer Resolution:** The kernel's only clock source is the 100 Hz PIT tick counter; there is no TSC- or HPET-backed cycle counter (Section 8.1). Every timing-based measurement in this paper inherits +-10 ms quantization, which is why several experiments (revocation propagation, Section 8.4) could only be bounded analytically rather than measured, and why others (`bench ipc`, Section 8.2) show run-to-run variance large enough that I report a median over five runs rather than a single number.
- **Simulation Telemetry:** The architecture simulation toolkit reads trace buffers generated during live execution. Because scheduler timing varies slightly between QEMU boots, the instruction and cycle counts are subject to minor run-to-run variations.
- **Adversary Assumption:** The validate-and-use race condition is demonstrated using a deterministic adversary. In a production system, this hazard would occur statistically based on timer interrupt alignment.
- **Static Task Ceiling:** `MAX_TASKS = 132` (Section 8.3) is a compile-time constant backed by statically-allocated arrays, not a dynamically-grown table. Every scalability claim in this paper (lock contention, scheduler throughput, distribution) is bounded by this ceiling and should be read as "true up to 132 tasks," not as a general scaling law.

Related Work

System	Capability Model	Preemption Handling	IPC Mechanism	Formal Verification
EROS [2]	Designator-based	Cooperative only	Rendezvous	No
seL4 [4]	Capability-based	Preemptive, proven non-interference	Async queued	Yes (machine-checked)
KeyKOS [6]	Capability-based	Preemptive (interrupt deferral)	Rendezvous	No
Capsicum [7]	Retrofitted (UNIX)	Host OS (Linux/FreeBSD)	Syscall-based	No
RunixOS	Capability-based	Preemptive + `CriticalWindow` RAII	Hybrid (sync + async)	No

Table 5: Comparison of capability-system designs along authority model, preemption handling, IPC, and verification.

Table 5 situates RunixOS among prior capability systems. EROS sidesteps the validate-and-use hazard entirely by never preempting mid-operation; its capability invocations are atomic because nothing else can run during them, the same property RunixOS's cooperative baseline has before preemption is introduced (Section 6.1). seL4 instead proves, via machine-checked refinement, that no preemption point in its kernel can observe or act on inconsistent capability state, which is a stronger guarantee than RunixOS provides but requires a formal proof effort far beyond this project's scope. KeyKOS handles the same underlying tension RunixOS faces, but does so by deferring hardware interrupts around sensitive kernel operations rather than by tracking an explicit software-visible window; RunixOS's `CriticalWindow` guard is closer in spirit to KeyKOS's interrupt deferral than to seL4′s proof, but is scoped narrowly to the IPC send path rather than the whole kernel, and is verified empirically (Section 6.5) rather than mechanically. The result, to my knowledge, is the first treatment of this hazard that is both (a) demonstrated in a live, preemptible kernel rather than argued about a cooperative or formally-modelled one, and

(b) closed with a minimal RAII-scoped deferral rather than a global interrupt-disable or a proof obligation.

RunixOS builds on early capability operating systems. The Dennis and Van Horn [1] capability model established the foundation for object-based access control. Levy [8] and Hardy [6] extended these designs to commercial computer architectures and persistent systems like KeyKOS. RunixOS differs by utilizing a modern, memory-safe systems language (Rust) and focusing on the synchronization hazards that preemption introduces to capability validation.

Compared to traditional microkernels like Mach [9], which suffered from IPC overhead, RunixOS adopts the minimal construction principles of L4 [3] to minimize kernel primitives. Unlike seL4 [4], which relies on formal verification to guarantee kernel invariants, RunixOS explores runtime mechanisms like `CriticalWindow` to resolve preemption concurrency hazards in capability routing. Additionally, Capsicum [7] retrofits capabilities into monolithic UNIX systems, whereas RunixOS adopts a clean-slate capability design.

Conclusion

This paper presented RunixOS, a capability-based, IPC-first microkernel operating system. I successfully demonstrated that capability-gated services can provide isolation, crash containment, persistence, and location-transparent routing under a minimal kernel. Furthermore, I identified a capability validate-and-use atomicity hazard introduced by preemptive scheduling and demonstrated that it must be closed centrally in the IPC send path. The `CriticalWindow` guard successfully defers preemption, restoring capability atomicity. Future development will focus on integrating physical storage and network drivers to transition the modelled persistence and distribution subsystems to physical hardware.

Appendix: Curriculum Alignment

This appendix maps WPI systems and architecture curriculum guidelines to the RunixOS implementation.

- **CS 3013 - Operating Systems:** Maps foundational operating system topics, including process management, memory paging, synchronization, and interrupts, to the core scheduler and IPC subsystems as detailed in the `CS3013_MAPPING.md` file.
- **CS 4513 - Distributed Computing Systems:** Maps advanced operating system concepts, including distributed resource allocation, filesystems, and performance evaluation, to the distribution substrate and service registry as detailed in the `CS4513_MAPPING.md` file.
- **CS 4515 - Computer Architecture:** Maps hardware architecture designs, including instruction pipelining, cache set associativity, and multi-core SMP, to the APIC driver and the architecture simulation toolkit as detailed in the `CS4515_MAPPING.md` file.

Bibliography

[1] J. B. Dennis and E. C. Van Horn, “Programming Semantics for Multiprogrammed Computations,” *Communications of the ACM*, vol. 9, no. 3, pp. 143–155, 1966.

[2] J. S. Shapiro, J. M. Smith, and D. J. Farber, “EROS: A Fast Capability System,” in *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*, 1999, pp. 170–185.

[3] J. Liedtke, “On Micro-Kernel Construction,” in *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*, 1995, pp. 237–250.

[4] G. Klein *et al.*, “seL4: Formal Verification of an OS Kernel,” in *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*, 2009, pp. 207–220.

[5] M. Bishop and M. Dilger, “Checking for Race Conditions in File Accesses,” *Computing Systems*, vol. 9, no. 2, pp. 131–152, 1996.

[6] N. Hardy, “KeyKOS Architecture,” *ACM SIGOPS Operating Systems Review*, vol. 19, no. 4, pp. 8–25, 1985.

[7] R. N. M. Watson, J. Anderson, B. Laurie, and K. Kennaway, “Capsicum: Practical Capabilities for UNIX,” in *Proceedings of the USENIX Security Symposium*, 2010, pp. 29–46.

[8] H. M. Levy, *Capability-Based Computer Systems*. Digital Press, 1984.

[9] M. Accetta *et al.*, “Mach: A New Kernel Foundation for UNIX Development,” in *Proceedings of the USENIX Summer Conference*, 1986, pp. 93–112.